ECE-C103 · Data Structures · Week 2

Algorithms and data

A study guide for the two introductory lectures and the Python lab.

> الدليل ده للمذاكرة بعد السكشن. ابدأ بالفكرة، جرّب المثال بإيدك، وبعدها اقرأ الكود. لو وقفت عند سطر، ارجع لقيم المتغيرات قبل السطر وبعده.

---

1. Describe the task before the code

An algorithm gives precise steps for completing a task. Start by stating the input and the required output. A data structure organizes the values those steps use. For example, `[8, 12, 5, 12]` is a list of current readings in amperes. “Visit each reading and count values at least 10” describes an algorithm. The answer is a count, not an ampere value.

> الـ data structure بتجاوب: البيانات مترتبة إزاي؟ والـ algorithm بتجاوب: هنعمل إيه بالبيانات؟ قبل ما تكتب، وضّح المدخلات والمطلوب. لما نعدّ قراءات بنرجّع عدد، مش مجموع القراءات.

A useful specification

Question	For our counter
What do we receive?	A finite list of numeric readings and a numeric limit.
What do we return?	How many values are greater than or equal to the limit.
What about repeated values?	Each occurrence counts separately.
What about an empty list?	Return 0.
May the data change?	This counter leaves the input list unchanged.

For the finite tasks in this week, require executable, unambiguous steps that terminate and produce the specified result. Correctness comes before efficiency. A slow correct method can still be an algorithm. Clear, concise pseudocode helps us inspect it, but algorithms can also be described in ordinary language or diagrams.

Assignment is a change in state

`a = b` reads the current value associated with `b`, then assigns it to `a`. It does not establish a permanent mathematical equation between the names. If we overwrite the old value of `a` before saving it, we cannot recover that value from `a`.

```
a = 7
b = 12
t = a
a = b
b = t
```

a	b	t
12	7	7

Put the saved value into b.

> علشان نبدّل 7 و12، بنحفظ قيمة a القديمة في t الأول. بعد كده نغيّر a، وبعدين نحط القيمة المحفوظة في b. t هنا اسم مؤقت، مش تعليمة خاصة في Python.

Lecture 1, PDF pp. 1–7; Lecture 2, PDF pp. 2–4.

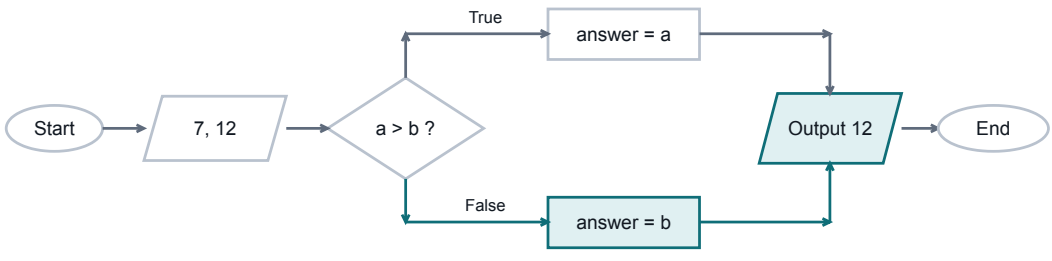
2. One algorithm, several representations

Words, pseudocode, a flowchart and Python can express the same decisions. Pseudocode has no single required syntax. Use indentation and precise verbs so another person can follow the steps. A flowchart uses arrows for order, an oval for start/end, a parallelogram for input/output, a rectangle for processing and a diamond for a decision. Label decision arrows True/False or Yes/No.

The larger value

For two numbers `a` and `b`, choose `a` when `a > b`; otherwise choose `b`. If the two values are equal, either is a correct larger value. If a task instead asks for the message “equal”, that is an additional output requirement.

```
IF a > b
    answer ← a
ELSE
    answer ← b
OUTPUT answer
```



a = 7, b = 12. The condition is False. Output: 12.

> الرسم والكود بيحكوا نفس القرار. عند المعين هنمشي في فرع واحد. لو a وb متساويين، شرط a > b يبقى False، لكن القيمة اللي هنرجّعها تفضل صحيحة.

Python syntax you need this week

Construct	Meaning	Example
`=`	Assign a value	`count = 0`
`==`	Test equality	`reading == target`
`>=`	Test “at least”	`reading >= limit`
`:` and indentation	Introduce and group a block	Indent the body by four spaces.
`def`	Define a function	The body runs when called.
`return`	Send a result back and end the call	`return count`
`print(...)`	Display a value	`print(answer)`

```
# Sheet 2, Q02
# PLAN
# HAVE: two numeric readings a and b.
# WANT: return the larger value, including when both are equal.
# TRY: 7 and 12 gives 12; 7 and 7 gives 7.
# STEPS: if a > b return a; otherwise return b.
# CHECK: both orders, equality and negative values.
def larger(a, b):
    # 1. choose the larger reading
    if a > b:
        return a
    return b

print(larger(7, 12))
print(larger(12, 7))
print(larger(7, 7))
print(larger(-5, -2))
```

▼ **Read the actual test output**

```
12
12
7
-2
```

Straight-line examples

The triangle algorithm receives base and height, calculates `base * height / 2`, then displays the result. With base 6 cm and height 4 cm, Python displays `12.0`; the area is 12 square centimetres. The sum example in Lecture 1 adds 529 and 254 and produces 783. Neither example needs a decision.

Profit and loss

Let `difference = income - cost`. A positive difference is profit. A negative difference indicates a loss of `-difference`. A zero difference means break-even. In Python, `elif` asks the next question only when the earlier condition was false.

```
difference = income - cost
if difference > 0:
    print("profit:", difference)
elif difference < 0:
    print("loss:", -difference)
else:
    print("break-even:", difference)
```

income	cost	difference
1000	800	200

profit: 200.

We use fixed numeric inputs while learning these ideas. A pseudocode step called “Read” means receive data; it does not force us to introduce keyboard-input parsing now.

Lecture 1, PDF pp. 7, 13–19. Python control flow.

3. Read a list, then count

A list has positions as well as values. For `readings = [8, 12, 5, 12]`, the length is 4 and its valid nonnegative indexes are 0, 1, 2 and 3. `readings[1]` is 12, not 1. Index 4 is outside this list. The two occurrences of 12 remain separate readings.



`range(len(readings))` provides the valid nonnegative indexes in order. The stop value is excluded. Keep this one loop form until you can trace it comfortably.

```
0 8
1 12
2 5
3 12
```

No indexes remain. The loop ends.

افصل بين i وبين readings[i]. الأولى رقم المكان، والتانية القيمة اللي فيه. جسم الـ loop بيتكرر لكل مكان. لو الـ list فاضية، مفيش indexes تتوزع، فالجسم مش هيتنفذ.

HAVE / WANT / TRY / STEPS / CHECK

1. **HAVE:** the readings and limit.
2. **WANT:** the count of readings at least the limit.
3. **TRY:** with limit 10, mark 8 no, 12 yes, 5 no, 12 yes. Two matches.
4. **STEPS:** start at zero, visit every index, increment for a match, return after the loop.
5. **CHECK:** test an empty list, equality, no matches and repeated matches.

```
# Sheet 2, Q04
# PLAN
# HAVE: a list of numeric readings and a numeric limit.
# WANT: number of readings greater than or equal to the limit.
# TRY: [8, 12, 5, 12], limit 10 gives 2.
# STEPS: start at zero; visit each position; add one for each match; return count.
# CHECK: empty list gives 0; equality counts; repeated readings count separately.
def count_at_least(readings, limit):
    # 1. start the count
    count = 0
    # 2. visit every position
    for i in range(len(readings)):
        # 3. count each qualifying reading
        if readings[i] >= limit:
            count = count + 1
    # 4. return after the entire list
    return count

print(count_at_least([8, 12, 5, 12], 10))
print(count_at_least([], 10))
print(count_at_least([10], 10))
print(count_at_least([2, 3], 10))
print(count_at_least([12, 12], 10))
```

`count = count + 1` reads the old count, adds one and stores the new count. It counts qualifying occurrences. It does not add the reading's value. The indentation of `return count` places it after the loop. An early return would stop before checking the remaining positions. Keeping it after the loop also returns 0 for an empty list.

i	count	limit
3	2	10

Return 2 after all readings.

▼ **Actual outputs for the five tests above**

```
2
0
1
0
2
```

A test you design yourself

Change the task to “strictly below the limit”. Write a new TRY example before changing the condition. Include a reading equal to the limit. That example distinguishes `<` from `<=`. Do Sheet 2 Q5 before checking your answer with the TA.

Lecture 2, PDF p. 9, traversal; original lab implementation. Python lists.

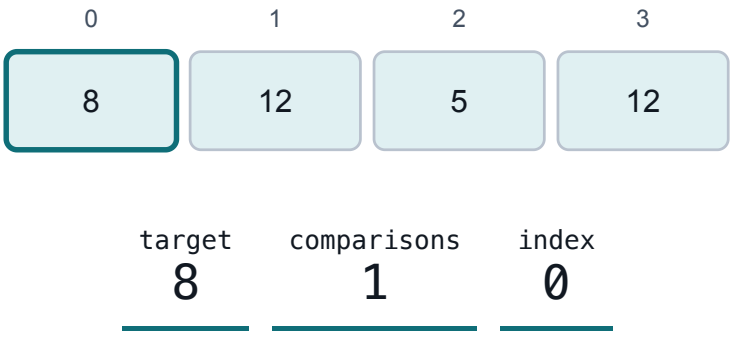
4. Explain time and space

Choose an input-size measure and an operation to count. Here `n` is the number of readings and one threshold test is one comparison. The counter makes exactly `n` threshold comparisons, whether all, some or none of the readings qualify. There are other operations too, but they do not change its linear growth under the usual unit-cost model.

n	comparisons / مقارنات
4	4
8	8
16	16
32	32

Cases depend on the algorithm

Linear search returns when it finds the first matching value. For a nonempty n-item list, finding the target first needs one equality comparison. Finding it last or failing to find it needs n comparisons. An empty list needs zero comparisons. The counter is different: it must inspect every reading even when the first one qualifies.



Target 8: 1 comparisons; returned index 0.

An average requires a distribution. If a target appears exactly once among four positions, and each position is equally likely, the average number of comparisons is `(1 + 2 + 3 + 4) / 4 = 2.5`. In general that model gives `(n + 1) / 2`. If unsuccessful searches can occur, include their probability and their n-comparison cost. A particular run never performs half a comparison.

> أفضل وأسوأ ومتوسط بتوصف اختيار المدخلات. مش كل algorithm بتقف أول ما تشوف تطابق. لازم تعرف هي بحث ولا عدّ كامل قبل ما تحسب التكلفة.

Growth patterns



n = 64. Bars share one linear scale within this frame; the scale changes between frames.

Pattern	Intuition and conditions
`O(1)`	A fixed amount of work independent of n, such as indexed access in an array model.
`O(log n)`	Repeatedly reduce the candidate range by a constant factor. Binary search requires sorted data.
`O(n)`	Visit all n readings once.
`O(n log n)`	Linear work across logarithmically many levels, as in merge sort.
`O(n²)`	For example, examine every ordered pair among n items, including self-pairs.

These formulas describe growth, not seconds. The bars use a common linear scale within each frame; the scale changes when n changes. Compare the printed values between frames. Not every nested loop is quadratic. Not every divide-and-conquer method is n log n. Quicksort has expected n log n time under suitable pivot assumptions but quadratic worst-case time. Searching a balanced search tree can be logarithmic; an unbalanced tree can have linear height.

O, Ω and Θ are bounds

Big O gives an asymptotic upper bound, Omega a lower bound, and Theta a tight bound up to constant factors for sufficiently large n. First select a cost function, such as worst-case comparisons. Then describe its growth using a bound. Theta does not mean average case.

3n	≤	3n + 2	≤	5n
Ω(n)		T(n)		O(n)

$$T(n) \in \Theta(n),\ n \geq 1$$

For `T(n) = 3n + 2` and `n ≥ 1`, the function lies between `3n` and `5n`, so it is `θ(n)`. It is also `O(n²)`, but that upper bound tells us less. Our full-scan counter takes `θ(n)` time in both its best and worst cases.

State the space convention

The input list already stores n readings. The counter adds a fixed number of working variables and no second n-item list. In the standard unit-cost/word model, its auxiliary space is `θ(1)`. Total space including the input is `θ(n)`. This classroom model does not count the bit growth of arbitrarily large Python integers. Creating a separate n-item copy would add linear auxiliary space; recursive calls can also use stack space.

Lecture 1, PDF pp. 8–12, 20–21. Definitions: NIST Big O and NIST Theta.

5. Types, contracts and structures

A data item may be one elementary field, such as a student ID. A group item combines related fields. A student record could group an ID, name and score. A data type defines a set of values and meaningful operations. A data structure organizes values to support operations such as access, search or update.

Basic values in Python

8 : int	8.5 : float	True : bool	"8" : str

`int` represents integers; `float` uses a finite floating-point representation and may approximate real-number values. `bool` has `True` and `False`. `str` represents Unicode text. Python has no separate character type: `"A"` is a one-character string. Text is not universally one ASCII byte per character. A collection such as a Python list can be built into a language, even when lecture classifications call collections “non-primitive”.

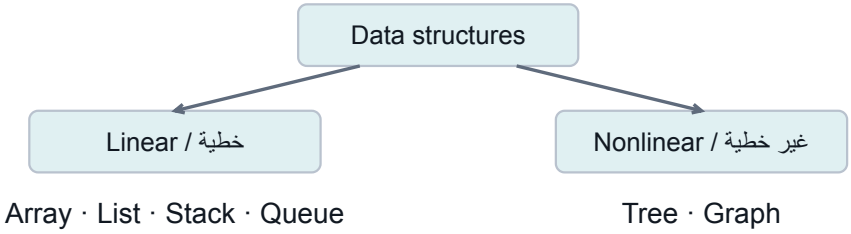
`8 + 2` produces 10, while `"8" + "2"` produces the string `"82"`. The values' types determine which operation the symbol performs.

Abstract data type, or ADT

An ADT specifies behavior independently of representation. A stack supports adding at the top and removing the most recently added item. An array-based list or linked nodes could implement that contract. The contract and the implementation answer different questions. You do not need to write a class to understand the contract.

الـ ADT اتفاق على العمليات ومعناها. التنفيذ بيختار مكان وطريقة التخزين. نفس الاتفاق ممكن نحققه بأكتر من data structure.

The family map



Structure	Relationship or behavior	Example
Array	Indexed, contiguous equal-type slots in the conventional model	Fixed positions for numeric measurements
Linked list	A sequence of nodes connected by links	Visit nodes by following next links
Stack	LIFO, last in first out	A stack of plates; undo history is a common application
Queue	FIFO, first in first out	Jobs served in arrival order
Tree	A hierarchy; a rooted tree gives each non-root node one parent	Folders inside folders
Graph	General connections; cycles may exist	Cities connected by roads
Hash table	Map keys to buckets and handle collisions	Look up a student record by ID

Linear means there is a sequence relationship. It does not require adjacent physical memory. A singly linked list has next links; a doubly linked list adds previous links. Those extra links consume space and must be updated consistently. A Python list is a resizable sequence of references, not a fixed homogeneous C++ array. Its referenced objects can have different types; we deliberately use numeric examples this week.



Remove C. LIFO: last added, first removed.



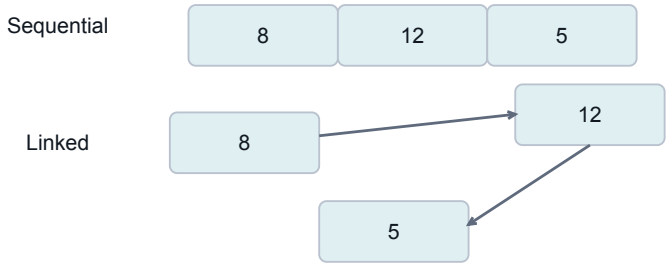
Remove A. FIFO: first added, first removed.

Removing from an empty stack or queue needs an explicitly defined behavior, such as an error or a chosen empty result. There is no item to return. A hash collision occurs when different keys map to the same bucket. Expected constant-time hashing needs assumptions about the hash function, collision handling and load.

Lecture 2, PDF pp. 3–8, 10. Python standard types.

6. Representation and operations

Sequential representation places slots next to one another. A known index supports direct access in the array model. Inserting or deleting in the middle may require shifting later entries. Linked representation stores connections between nodes, which need not be adjacent in memory. Reaching a position may require following many links.



If the necessary node or predecessor is already known, a linked-list edit can change only a fixed number of links. If it must first be found, that search adds cost. Appending to a singly linked list is constant-time when a maintained tail reference is available; otherwise reaching the tail can require a full traversal. Neither representation is always preferable.

> ما تقولش linked أسرع في الإضافة من غير ما تسأل: وصلت للمكان أصلًا ولا لسه هتبحث؟ الوصول والتعديل عمليتين، وممكن تكلفة الوصول تكون هي الأكبر.

Six operations



Merge sorted [5, 8] and [10, 12] into one sorted sequence.

Operation	Meaning	Question to specify
Traversal	Visit elements for processing	In what order? Visit each once?
Search	Locate matching data	First match, all matches, or existence only?
Insertion	Add a new element	At which position or under which ordering rule?
Deletion	Remove an element	By index, first value match, or all matches?
Sorting	Arrange elements by a defined order	Ascending or descending? Which field?
Merging	Combine collections	Must sorted inputs produce a sorted result?

The animated examples restart from the original data for each operation. For sorted merging, combine `[5, 8]` with `[10, 12]` while preserving order. Simply concatenating arbitrary sorted lists is not always a sorted merge. Deleting the first 12 from `[8, 12, 5, 12]` requires finding its position and removing that occurrence. The other 12 remains.

Lecture 2, PDF pp. 8–9. Open Data Structures, linked lists.

7. Recognize design strategies

These are introductory ideas from Lecture 1. Implementation of these strategies is outside this week's Python exercises.

Idea	Mechanism	Small example or limit
Brute force	Try candidates directly	Search each list position for a target. This can be reasonable for small inputs.
Divide and conquer	Split into smaller instances, solve them, combine results	Merge sort sorts halves then merges them.
Recursion	A procedure calls itself on a smaller problem	A countdown reaches a base case at zero. Pending calls can use extra memory.
Greedy	Choose a locally attractive action	Choosing the largest coin first does not always minimize the number of coins.
Dynamic programming	Reuse results of repeated smaller problems	Store the best coin count for smaller amounts before solving larger amounts.
Tree traversal	Visit a structure's nodes in a defined order	Process files in a folder tree.
State-space search	Explore states and permitted moves toward a goal	Consider possible moves from a maze junction.

With unlimited coins of values 1, 3 and 4, amount 6 can use `4 + 1 + 1` or `3 + 3`. The greedy first choice of 4 leads to three coins, but the second combination uses two. A design strategy needs a reason it fits the task; its name is not a guarantee of correctness or optimality.

الطرق دي أفكار تصميم، مش أسماء نلصقها بأي كود. مثال العملات يوضّح إن أحسن اختيار دلوقتي ممكن ما يكونش أحسن حل كامل. وفي recursion لازم المشكلة تصغر وفيه حالة توقف واضحة.

Lecture 1, PDF pp. 4, 22; original coin example checked by exhaustive small-case computation.

8. Check your understanding

Answer first, then open the explanation. Use these questions to locate what needs more practice.

▼ 1. Why is return count after the loop?

We need all readings before returning the final count. An early return may miss later matches. Placing the return after the loop also handles an empty input with count still 0.

▼ 2. What does count_at_least([10], 10) return?

1. “At least” includes equality. The comparison is >=.

▼ 3. Does a repeated 12 count once or twice?

Each occurrence is a separate reading. In [8, 12, 5, 12], the two 12 values occupy different positions, so both qualify at limit 10.

▼ 4. What changes when n doubles for the full scan?

The number of threshold comparisons doubles. Extra working-variable count stays fixed in the unit-cost model.

▼ 5. Is Θ the symbol for average case?

No. It expresses a tight asymptotic bound on the selected cost function. Best, worst and average describe how we choose or distribute inputs.

▼ 6. Which factors matter in choosing a structure?

The required operations, correctness, time and memory. Hardware can affect practical timings, but this week's abstract analysis focuses on time and space. Lecture 2 self-evaluation Q5 is intended to select “Space and Time”.

▼ 7. Is a type a set of values together with operations?

Yes. Integers with arithmetic are one example. An ADT specifies behavior independently of representation. A data structure supplies a concrete organization.

▼ **8. Give a structured type and classify a graph**

An array or record is a structured example. A graph is nonlinear. A linked list is linear even if its nodes are physically separated.

▼ **9. Explain LIFO and FIFO for arrivals A, B, C**

LIFO removes C first. FIFO removes A first. The allowed removal rule is part of the ADT contract.

▼ **10. Must linked representation always be preferred?**

No. It depends on access and update patterns. Linked edits can be cheap once the location is known, but following links to find that location can cost linear time.

▼ **11. Is “int” universally a fixed-size storage keyword?**

No. In C/C++ it names an integer type, but Python's int supports arbitrary precision subject to available memory. Lecture 2 Q11 uses language-dependent wording; do not generalize a fixed byte count to Python.

How to use this guide

For the core lab, revisit Chapters 1–4 and finish Sheet 2 Q1–Q5. Chapters 5–7 cover the lecture continuation and its vocabulary. Keep your code files under `DSA/sheet02`, and follow the actual deadline and submission channel announced by the TA.

Source note

The professor's two PDFs set the topic coverage and remain unchanged in the course materials. This guide uses PDF page numbers, which differ from some printed slide numbers. The lab clarifies language-dependent type claims and the distinction between asymptotic bounds and input cases. Diagrams are original teaching illustrations, not screenshots or measurements of physical memory.